

R Lab3. Manejo de secuencias en R

Bioinformática. Ingeniería Biomédica

Las secuencias ya sean de ácidos nucleicos o de aminoácidos es una parte central en el análisis de datos bioinformáticos. Su análisis es la tarea más básica en bioinformática.

Se representan en términos de una secuencia de caracteres. Desde el punto de vista de los datos, podemos entender el dogma central de la biología molecular como la conversión de caracteres de un tipo en otro tipo. Entonces la secuencia de DNA va a ser fundamental para diversas aplicaciones que van desde la comparación de biomoléculas para el estudio de la evolución o de mutaciones, hasta la identificación de sitios interesantes o la predicción de estructuras tridimensionales.

Vamos a analizar las propiedades de las secuencias tratando de comprender el mecanismo subyacente de la transformación de un tipo de información a otro.

Leyendo y escribiendo ficheros FASTA

El formato 'FASTA' es un formato muy simple basado en texto que se utiliza para representar secuencias, ya sea de nucleótidos o de aminoácidos. En los dos casos, tanto un nucleótido como un aminoácido se representa por una única letra. También podemos encontrar algunos símbolos para representar

- un hueco (gap), de longitud indeterminada: -
- parada en la traducción: *
- se desconoce el nucleótido o aminoácido que hay en esa posición: X.

El fichero comienza con el símbolo > (mayor que) al que sigue una línea descriptiva y opcional de la secuencia. Esta línea contiene el identificador de la secuencia (sin espacio entre el símbolo > y la primera letra del identificador). En las secuencias que podemos obtener de diferentes bases de datos, esta línea identifica la base de datos, el identificador de secuencia, entre otras, que se separan con el caracter |.

En la siguiente línea empieza la secuencia de bases o aminoácidos. Se recomienda que las líneas tengan una longitud máxima de 80 caracteres y puede contener todas las filas que se necesiten. La secuencia termina si aparece otra línea comenzando con el símbolo >, que indica el comienzo de otra nueva secuencia. Un ejemplo simple de una secuencia en el formato 'FASTA' puede ser:

```
>gi|5524211|gb|AAD44166.1| cytochrome b [Elephas maximus maximus]
LCLYTHIGRNIYYGSYLYSETWNTGIMLLITMATAFMGYVLPWQMSFWGATVITNLFSaipyigtNLV
EWIWGGFSVDKATLNRFFAFHFILPFTMVALAGVHLTFLHETGSNNPLGLTSDSDKIPFHPYYTIKDFLG
LLILILLLLLLALLSPDMLGDPDNHMPADPLNTPLHIKPEWYFLFAYAILRSVPNKLGGVLALFLSIVIL
GLMPFLHTSKHRSMMLRPLSQALFWTLTMDLLTLTWIGSQPVEYPYTIIGQMASILYFSIILAFPLIAGX
IENY
```

Es el formato más popular para almacenar secuencias por lo que va a ser importante que seamos capaces desde R tanto de importar como de exportar ficheros en este formato.

Leer un fichero FASTA

Vamos a manejar la secuencia de RNA polimerasa de la Mycobacterium tuberculosis (actinobacteria) y de la Escherichia coli (Proteobacteria). Tenemos la información en el fichero 'GC_Bacterias.fasta' con las dos secuencias con las que vamos a trabajar.

Para leer un fichero FASTA utilizamos la función `read.fasta()` del paquete `seqinr` de Bioconductor. Esta función tiene un argumento, `file`, en donde se indica el nombre del archivo a leer.

1. Lee el fichero 'GC_Bacterias.fasta' y crea los objetos `myActino` y `myProteo` con la información de las secuencias correspondientes.

Escribir un fichero FASTA

Vamos a crear un fichero FASTA con la información de las secuencias que nos interesan. Podemos acceder a cada secuencia utilizando la función `seqinr::getSequence()`. Por ejemplo, para la primera secuencia,

```
myseq <- getSequence(myActino)
```

2. Utiliza la función `write.fasta()` para crear un fichero 'fasta' con esta información. Esta función tiene los siguientes argumentos:

- En el primer argumento debemos indicarle la secuencia (o secuencias),
- el segundo argumento el nombre(s) que podemos obtener con la función `seqinr::getName()` y
- en el argumento `file.out` debemos indicar el nombre del fichero, que tendrá extensión 'fasta'.

3. Tenemos la secuencia en letras minúsculas, utiliza la función `base::toupper()` para escribir la secuencia con letras mayúsculas.

4. Si quisiésemos incluir varias secuencias en el mismo fichero podríamos indicarlo en los dos primeros argumentos. El argumento `sequences` debe ser una lista con tantos elementos como secuencias a incluir en el fichero, y `names` un vector de caracteres con los nombres correspondientes.

Escribe un archivo 'fasta' con las dos secuencias de interés.

Análisis del contenido de una secuencia

El paquete `seqinr` también dispone de diferentes métodos que podemos utilizar para obtener información básica como, por ejemplo, la frecuencia de nucleótidos o aminoácidos, o el contenido GC (Guanina / Citosina). Esto servirá para establecer conclusiones importantes tales como, entre otras:

- Si una secuencia tiene muchas bases repetidas, tipo 'AAAAAA', a priori podemos pensar que no será una secuencia muy interesante puesto que tendrá un bajo contenido de información.
- El contenido de una secuencia puede ayudarnos a determinar propiedades de la molécula completa como la hidrofobicidad.
- En ciertos genomas, especialmente entre las bacterias, hay diferencias en el contenido de GC. Por ejemplo, algunas actinobacterias pueden tener más del 70 por ciento de GC, mientras que algunas proteobacterias (que incluyen una gran variedad de patógenos) puede tener menos del 20 por ciento de GC.
- Además, el contenido de GC también se utiliza para predecir la temperatura de hibridación durante los experimentos de PCR.

Estos aspectos, entre otros, hacen que el análisis del contenido de una secuencia sea importante. Veamos como llevarlo a cabo a partir de las secuencias obtenidas con `seqinr`.

1. Calcula el número de apariciones de cada base en las secuencias utilizando la función genérica `table()`. Calcula también las frecuencias relativas.

2. Utiliza la función `seqinr::GC()` para obtener el contenido de G+C en las secuencias. ¿En que escala obtienes ese valor?

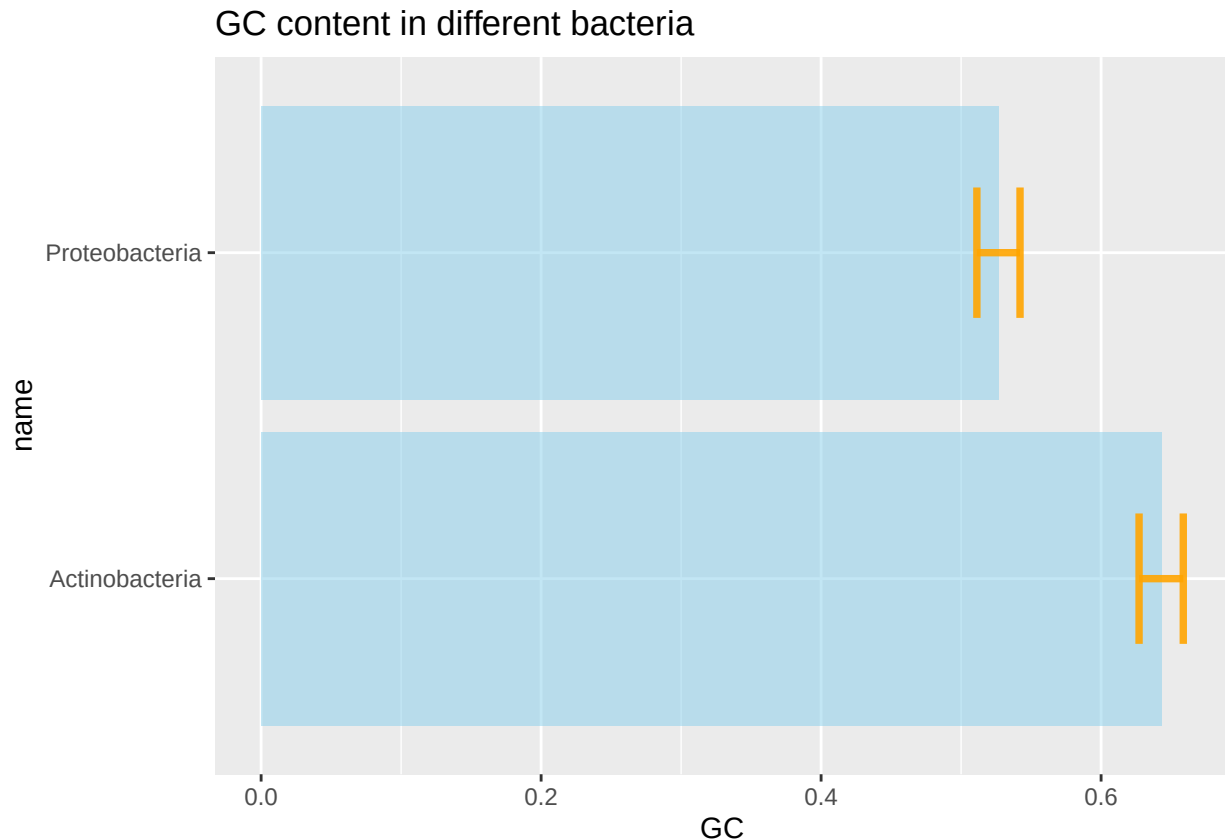
3. Calcula un intervalo de confianza para esas proporciones. Recuerda, que para una proporción la expresión del intervalo de confianza es,

$$\hat{p} \pm Z_{1-\frac{\alpha}{2}} \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

Utiliza la función `qnorm()` para calcular los percentiles de una normal.

¿Hay diferencias significativas en el contenido de GC entre tipos de bacterias?

4. Crea un gráfico de barras para mostrar el porcentaje de GC junto con sus intervalos de confianza.



5. Además de las frecuencias de cada nucleótido individual, nos puede interesar la frecuencia de “palabras” más largas. Podemos obtener frecuencias de conjuntos de nucleótidos con la función `seqinr::cont()`. Esta función tiene un argumento `wordsize` para indicar el tamaño de las subsecuencias a contar. Obtén las frecuencias de tripletes de nucleótidos (codón).

6. Podemos utilizar el estadístico ρ (rho) para comprobar si alguna “palabra” está sobre o infra representada en la secuencia de interés. Que esté sobre-representada significa que su frecuencia es mayor de la que se esperaría por azar, mientras que estará infra-representada si su frecuencia es menor de lo que se espera. Para dos nucleótidos esta medida se puede calcular como,

$$\rho(x, y) = \frac{f_{xy}}{(f_x \hat{u} f_y)}$$

El numerador f_{xy} representa la frecuencia relativa observada y el denominador $f_x \hat{u} f_y$ la que se esperaría por azar. Cuando $\rho = 1$ la frecuencia observada es igual a la esperada. Un valor de $\rho < 1$ indica infra-representación y $\rho > 1$ sobre-representación. El valor > 1 se puede interpretar como cuantas veces más está sobre-representado el par correspondiente. Si $\rho < 1$ podemos interpretar el inverso ($\frac{1}{\rho}$) e interpretarlo como cuantas veces más se espera que esté ese par respecto de lo observado.

Podemos utilizar la función `seqinr::rho()` para hacer estos cálculos,

```
seqinr::rho(myProteo,wordsize=2)
```

```
##
##      aa      ac      ag      at      ca      cc      cg      ct
## 1.2830596 1.0329297 0.8174443 0.8590758 0.8217359 0.8933309 1.2322933 1.0442815
##      ga      gc      gg      gt      ta      tc      tg      tt
## 1.1239859 0.9528779 0.7319573 1.2416023 0.7299919 1.1447772 1.2665174 0.8154573
```

Podríamos pensar en construir un contraste de hipótesis para ver si esta sobre o infra-representación es estadísticamente significativa.

Alineamiento de pares de secuencias

Por alineamiento de pares de secuencias nos referimos a la manera óptima de colocar las dos secuencias para identificar regiones que sean similares entre sí. Necesitaremos encontrar el alineamiento óptimo.

Alinear dos secuencias utilizando el paquete Biostrings

Vamos a alinear dos secuencias utilizando el paquete Biostrings. Instala y carga la librería.

1. Necesitamos dos secuencias. En este ejemplo, vamos a inventarnos dos secuencias de DNA muy sencillas,

```
sequence1 <- "GAATTCGGCTA"
sequence2 <- "GATTACCTA"
```

Notad que no necesariamente tienen porque tener la misma longitud.

2. Definimos el sistema de puntuación. Con la función nucleotideSubstitutionMatrix() asignamos la puntuación para los *match* y *mismatch*.

```
(myScoringMat <- nucleotideSubstitutionMatrix(match = 1, mismatch = -1, baseOnly = TRUE))
```

```
## Warning in .call_fun_in_pwalign("nucleotideSubstitutionMatrix", ...): nucleotideSubstitutionMatrix()
## call pwalign::nucleotideSubstitutionMatrix() to get rid of this
## warning.
```

```
##      A  C  G  T
## A   1 -1 -1 -1
## C  -1  1 -1 -1
## G  -1 -1  1 -1
## T  -1 -1 -1  1
```

2.1. *Penalizaciones por huecos (gap penalties)*. Sirven para indicar al algoritmo si queremos favorecer la aparición de *gaps* en el alineamiento o no. Normalmente se pueden modificar dos tipos de penalizaciones: el de creación de un nuevo hueco (*gap open*) y el de extensión del hueco (*gap extend*).

```
gapOpen <- 2
gapExtend <- 1
```

3. Utilizamos la función pairwiseAlignment() para llevar a cabo un alineamiento global (argumento type="global"),

```
(myAlignment <- pwalign::pairwiseAlignment(sequence1, sequence2, substitutionMatrix = myScoringMat,
                                           gapOpening = gapOpen, gapExtension = gapExtend,
                                           type="global", scoreOnly = FALSE))
```

```
## Global PairwiseAlignmentsSingleSubject (1 of 1)
## pattern: GAATTCGGCTA
## subject: GATTAC--CTA
```

```
## score: 1
```

Si `scoreOnly=TRUE` solo devuelve la puntuación del alineamiento. `type="global"` para utilizar el algoritmo de *Needleman-Wunsch*.

Pares de proteínas

Para el caso de secuencias de aminoácidos, el paquete **Biostrings** dispone de las matrices de sustitución en forma de *datasets*,

```
data(package="Biostrings")
```

Vamos a utilizar la misma función `pairwiseAlignment()` para alinear dos secuencias de aminoácidos,

```
sequence1 <- "PAWHEAE"  
sequence2 <- "HEAGAWGHE"
```

La función `pairwiseAlignment()` necesita los siguientes argumentos:

- Las dos secuencias a alinear
- `substitutionMatrix` la matriz de sustitución. En este caso vamos a seleccionar 'BLOSUM62'
- las penalizaciones por los gaps: `gapOpening` y `gapExtension`. Vamos a mantener las mismas que las que utilizamos para las secuencias de nucleótidos.
- `type`, el tipo de alineamiento: `global` o `local`.
- `scoreOnly=FALSE` para que muestre el alineamiento además de la puntuación.

1. Alinea globalmente las dos secuencias.

2. Alinea localmente las dos secuencias.

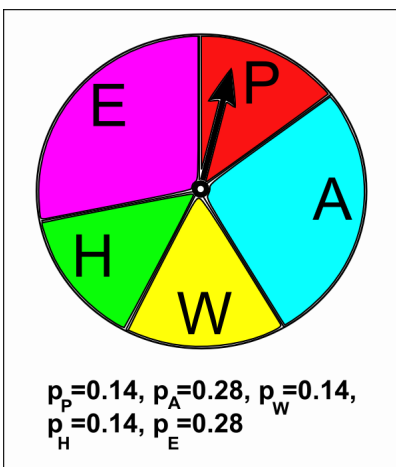
Significación estadística de un alineamiento global

Con el alineamiento global de las secuencias 'PAWHEAE' y 'HEAGAWGHEE', hemos visto que tienen cierta similitud, pero ¿es estadísticamente significativa? Es decir, ¿es esta alineación mejor de lo que esperaríamos por azar?

El algoritmo de alineación de *Needleman-Wunsch* siempre producirá una alineación global incluso con dos secuencias de proteínas no relacionadas, aunque la puntuación de alineación sería baja. Por lo tanto, la pregunta que nos tendríamos que hacer es: ¿la puntuación de nuestro alineamiento es significativamente mejor de lo esperado por azar entre dos secuencias de la misma longitud y composición de aminoácidos?

Por lo tanto, para evaluar si la puntuación de nuestra alineación entre la secuencia de proteínas 'PAWHEAE' y 'HEAGAWGHEE' es estadísticamente significativa, el primer paso será construir algunas secuencias aleatorias que tengan la misma composición de aminoácidos y la misma longitud que una de nuestras dos secuencias iniciales, por ejemplo elegimos 'PAWHEAE'.

¿Cómo podemos obtener secuencias aleatorias de la misma composición y longitud de aminoácidos que la secuencia 'PAWHEAE'? Una forma es generar secuencias utilizando un **modelo multinomial** en el que las probabilidades de los diferentes aminoácidos se establecen para ser iguales a sus frecuencias en la secuencia 'PAWHEAE'. Es decir, las probabilidades de 'P', 'W' y 'H' serían 0.1428571 (1/7); y las de 'A' y 'E' 0.2857143 (2/7). Las probabilidades de los otros 15 aminoácidos se establecen en 0. Para generar una secuencia de longitud 7 según un modelo multinomial elegimos la letra para cada posición en la secuencia de acuerdo con esas probabilidades, sería como girar la ruleta del dibujo 7 veces. Tendremos que generar un número grande de veces una secuencia de longitud 7 según este modelo.



Podemos utilizar la siguiente función, `generateSeqsWithMultinomialModel()` para genera un número X de veces la secuencia `inputsequence`,

```
generateSeqsWithMultinomialModel <- function(inputsequence, X) {
  require("seqinr") # This function requires the seqinr package.

  # Change the input sequence into a vector of letters
  inputsequencevector <- s2c(inputsequence)

  # Find the frequencies of the letters in the input sequence "inputsequencevector":
  mylength <- length(inputsequencevector)
  mytable <- table(inputsequencevector)

  # Find the names of the letters in the sequence
  letters <- rownames(mytable)
  numletters <- length(letters)
  # Make a vector to store the probabilities of letters
  probabilities <- numeric()
  for (i in 1:numletters) {
    letter <- letters[i]
    count <- mytable[[i]]
    probabilities[i] <- count/mylength
  }

  # Make X random sequences using the multinomial model with probabilities "probabilities"
  seqs <- numeric(X)
  for (j in 1:X) {
    seq <- sample(letters, mylength, rep=TRUE, prob=probabilities) # Sample with replacement
    seq <- c2s(seq)
    seqs[j] <- seq
  }

  # Return the vector of random sequences
  return(seqs)
}
```

La función devuelve X secuencias en un vector de X elementos.

1. Genera 1000 secuencias aleatorias según el modelo multinomial en el que las probabilidades de cada letra se establecen según la secuencia 'PAWHEAE'.

```
## [1] "AHAEAPH" "AEAPAAA" "EEWAPAE" "WEWWWPP" "EEAAAAEA" "EPHEAEA" "AEEHEEW"  
## [8] "HEEAAHA" "EPWEAAA" "HEPAEHA"
```

2. Lleva a cabo el alineamiento global de cada una de estas secuencias aleatorias con 'HEAGAWGHEE', almacenando la puntuación del alineamiento obtenido. Utiliza la matriz de sustitución BLOSUM50 y -2 y -8 las penalizaciones por abrir un *gap* y por extenderlo, respectivamente.
3. ¿Cuál es la puntuación del alineamiento entre las secuencias originales en esas condiciones?
4. Estima la probabilidad de obtener una puntuación mayor que la observada (alineamiento real) cuando no hay relación entre las secuencias (alineamientos con secuencias aleatorias).